Practical No.: 2

Name: More Vaishnavi Dadaso

Roll No.: 269

Problem Statement: Implement a stack as an abstract data type using a singly linked list, and use this ADT to convert infix expressions to postfix and prefix.

Code:

```
#include <cstring>
#include <algorithm>
using namespace std;

struct Node
{
   char data;
   Node* next;
};
class Stack
{
   Node* top;
public:
   Stack()
   {
      top = NULL;
   }
     void push(char x)
   {
      Node* temp = new Node;
      temp->data = x;
      temp->next = top;
      top = temp;
```

```
    }
      char pop()
    {
      if (top == NULL)
        return '#';
      Node* temp = top;
      char ch = temp->data;
      top = top->next;
      delete temp;
      return ch;
    }
      char peek()
    {
      if (top == NULL)
        return '#';

      return top->data;
    }
       bool isEmpty()
    {
      return top == NULL;
    }
};
int priority(char op)
{
    if (op == '^')
      return 3;

    if (op == '*' || op == '/')
```

```
        return 2;

    if (op == '+' || op == '-')
        return 1;

    return 0;
}
bool operand(char ch)
{
    if ((ch >= 'A' && ch <= 'Z') ||
        (ch >= 'a' && ch <= 'z') ||
        (ch >= '0' && ch <= '9'))
    {
        return true;
    }
return false;
}
string infixToPostfix(string exp)
{
    Stack s;
    string result = "";
    for (int i = 0; i < exp.length(); i++)
    {
        char ch = exp[i];

        if (operand(ch))
        {
            result += ch;
        }
```

```
      else if (ch == '(')
      {
         s.push(ch);
      }
      else if (ch == ')')
      {
         while (!s.isEmpty() && s.peek() != '(')
         {
            result += s.pop();
         }

         s.pop();
      }
      else
      {
         while (!s.isEmpty() &&
               priority(s.peek()) >= priority(ch))
         {
            result += s.pop();
         }
        s.push(ch);
      }
   }
    while (!s.isEmpty())
   {
      result += s.pop();
   }
     return result;
}
```

```
string infixToPrefix(string exp)
{
   reverse(exp.begin(), exp.end());
  for (int i = 0; i < exp.length(); i++)
   {
      if (exp[i] == '(')
         exp[i] = ')';

      else if (exp[i] == ')')
         exp[i] = '(';
   }
   string postfix = infixToPostfix(exp);
   reverse(postfix.begin(), postfix.end());
   return postfix;
}
int main()
{
   string exp;
   cout << "Enter Infix Expression : ";
   cin >> exp;
    cout << "\nPostfix : " << infixToPostfix(exp);
   cout << "\nPrefix : " << infixToPrefix(exp);
   return 0;
}
```

Output:

```
Enter Infix Expression : (A+B)*(C+D)+E

Postfix : AB+CD+*E+
Prefix : +*+AB+CDE
```